MISA®
TIN CẬY - TIỆN ÍCH - TẬN TÌNH

Đảm bảo hiệu năng ở tầng Front-end

KHÁCH THÍCH - KHÁCH YÊU - KHÁCH CHIA SẺ

Mục tiêu bài học

- Nắm bắt được một số các best practice khi code frontend giúp tối ưu hiệu năng sản phẩm, tăng trải nghiệm cho người sử dụng.

Nội dung bài học

1. Tối ưu tốc độ tải trang
2. Tối ưu hiệu năng trong Vue
3. Tối ưu code Javascript
4. Tối ưu request API

Tối ưu tốc độ tải trang

1. Code Splitting và Lazy loading components

- **Code Splitting** là kỹ thuật chia nhỏ mã nguồn thành các phần nhỏ hơn để tải dần dần khi người dùng cần thay vì tải cùng 1 lúc.
- **Lazy loading components** là kỹ thuật chỉ tải các phần của ứng dụng khi người dùng cần đến chúng, thường được áp dụng cho các route.

➡ Trong Vue chúng ta có thể cấu hình **Vue Router** để lazy load các route bằng cách **sử dụng dynamic import**. Điều này giúp tách riêng từng trang hoặc component và chỉ tải chúng khi cần.

✖ KHÔNG NÊN

```
import { createRouter, createWebHistory } from 'vue-router'
import EmployeeView from '../views/dictionary/EmployeeList.vue'
import CustomerView from '../views/dictionary/CustomerList.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/employee',
      name: 'employee',
      component: EmployeeView
    },
    {
      path: '/customer',
      name: 'customer',
      component: CustomerView
    }
  ]
})
```

✔ NÊN

```
import { createRouter, createWebHistory } from 'vue-router'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/employee',
      name: 'employee',
      component: () =>
        import(/* webpackChunkName: "di" */ '../views/dictionary/EmployeeList.vue')
    },
    {
      path: '/customer',
      name: 'customer',
      component: () =>
        import(/* webpackChunkName: "di" */ '../views/dictionary/CustomerList.vue')
    }
  ]
})
```

1. Code Splitting và Lazy loading components

- Trong Vue, webpack có hỗ trợ cấu hình `Code Splitting` giúp chia nhỏ các phần mã thành các `chunks`, thông qua thuộc tính **`webpackChunkName`**, cấu hình import các component có cùng `webpackChunkName` thì sẽ được tạo thành 1 `chunk` để được tải về cùng 1 lúc.
- Thường cấu hình `webpackChunkName` để gom các component thành 1 file trong các trường hợp sau:
  - Các component của cùng 1 chức năng

    (VD: các component danh sách, chi tiết, tìm kiếm,.. của tính năng bán hàng được cấu hình cùng webpackChunkName: "sale")
  - Các chức năng độc lập sử dụng ở nhiều nơi như popup
  - Các component lớn hoặc ít được sử dụng
- Sử dụng **`webpack-bundle-analyzer`** để hỗ trợ phân tích kích thước các file `chunk` mà webpack tạo ra để đánh giá xem cấu hình gom như nào cho hợp lý.

```
// vue.config.js
const { BundleAnalyzerPlugin } = require('webpack-bundle-analyzer');

module.exports = {
  configureWebpack: {
    optimization: {
      splitChunks: {
        chunks: 'all',
      }
    },
    plugins: [
      new BundleAnalyzerPlugin({
        analyzerMode: 'static',
        openAnalyzer: true,
        reportFilename: 'bundle-report.html'
      })
    ]
  }
};
```

2. Sử dụng keep-alive để cache components

`keep-alive` dùng để cache các component được bao bọc bên trong nó, giúp giảm thiểu việc tải lại dữ liệu khi người dùng điều hướng qua lại giữa các trang. Lợi ích của keep-alive:

- **Tối ưu hóa hiệu suất:** Tránh việc tạo lại component từ đầu, giúp giảm tải công việc cho trình duyệt và tăng tốc độ phản hồi của ứng dụng.
- **Giữ nguyên trạng thái:** Trạng thái của component, bao gồm dữ liệu, biến local, và trạng thái DOM, được giữ lại, giúp duy trì trải nghiệm người dùng liên tục.

keep-alive thường sử dụng với Vue Router, cách dùng là sử dụng các thuộc tính `include`, `exclude`, `max` để kiểm soát những component nào sẽ được cache và không cache:

- include: cấu hình danh sách các component được cache
- exclude: cấu hình danh sách các component không được cache
- max: giới hạn số lượng component sẽ được cache

```
<template>
  <div id="app">
    <keep-alive :include="['customer', 'employee']">
      <RouterView />
    </keep-alive>
  </div>
</template>
```

```
<template>
  <div id="app">
    <keep-alive :exclude="['users', 'roles']">
      <RouterView />
    </keep-alive>
  </div>
</template>
```

```
<template>
  <div id="app">
    <keep-alive :max="5">
      <RouterView />
    </keep-alive>
  </div>
</template>
```

2. Sử dụng keep-alive để cache components

Trong Vue, component có sử dụng **keep-alive** thì có thể đăng ký các lifecycle hooks `onActivated` và `onDeactivated` để quản lý trạng thái.

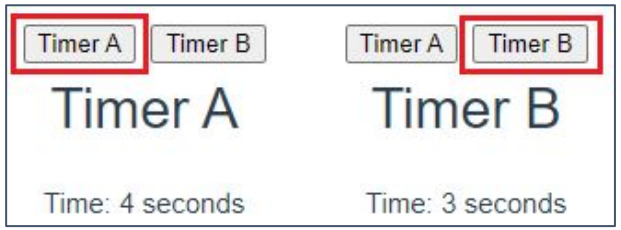
Ví dụ sau có component Demo.vue sử dụng `keep-alive` để giữ lại các component TimerA.vue và TimerB.vue khi chúng bị chuyển đổi. Trên từng component TimerA.vue và TimerB.vue sẽ:

- Sử dụng onActivated để bắt đầu bộ đếm thời gian (timer) khi component được kích hoạt
- Sử dụng onDeactivated để dừng timer khi component bị hủy kích hoạt.
- timer sẽ tăng giá trị time mỗi giây khi component đang hoạt động.

```vue
<!-- Demo.vue -->
<template>
  <div id="appdemo">
    <button @click="currentView = 'TimerA'">Timer A</button>
    <button @click="currentView = 'TimerB'">Timer B</button>

    <keep-alive>
      <component :is="currentView" />
    </keep-alive>
  </div>
</template>
```

Hình UI bên dưới sẽ thấy khi bấm nút để vào component TimerA thông tin Time chạy đên 4s, bấm chuyển sang component TimerB thông tin Time chạy đến 3. Sau đó chuyển lại về component TimerA thì Time tăng tiếp từ 4.




```vue
<!-- TimerA.vue -->
<template>
  <div>
    <h1>Timer A</h1>
    <p>Time: {{ time }} seconds</p>
  </div>
</template>
```

```vue
const time = ref(0);
let timer;

const startTimer = () => {
  timer = setInterval(() => {
    time.value++;
  }, 1000);
};

const stopTimer = () => {
  clearInterval(timer);
};

onActivated(() => {
  startTimer();
  console.log('Timer A activated');
});

onDeactivated(() => {
  stopTimer();
  console.log('Timer A deactivated');
});
```

2. Sử dụng keep-alive để cache components

Các lưu ý khi sử dụng keep-alive

Khi sử dụng keep-alive thì các component được cache trong bộ nhớ, sẽ gây **tốn RAM trình duyệt** do đó cần lưu ý

KHÔNG NÊN:

- Cache lại các component nặng
- Cache lại các component không cần giữ lại trạng thái
- Component phụ thuộc vào tài nguyên bên ngoài cần được làm mới

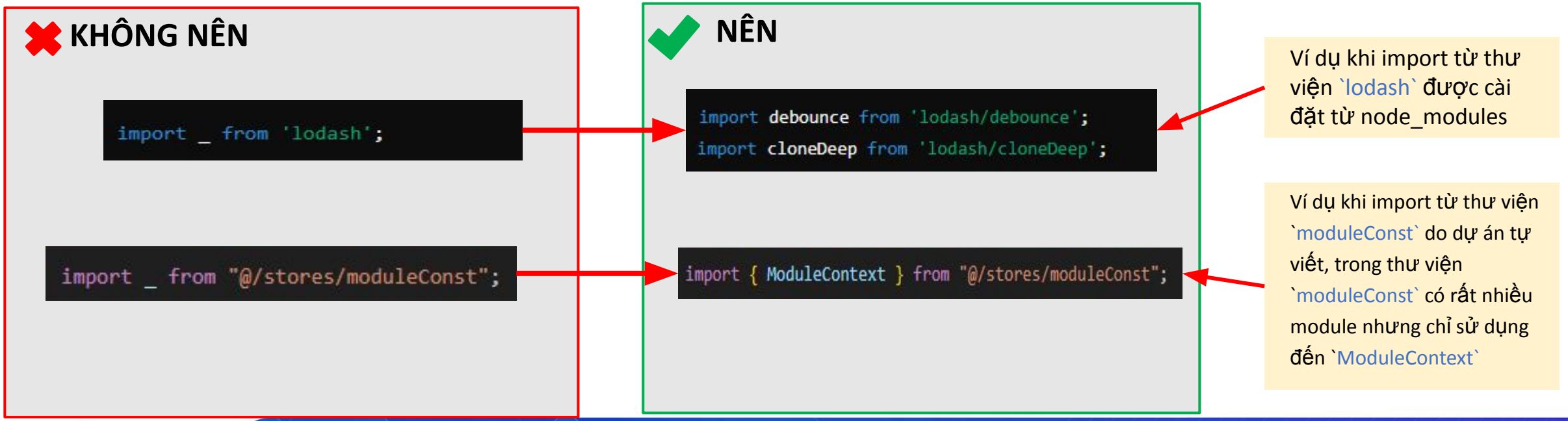
NÊN:

- Chỉ cache các component **quan trọng**, **thường xuyên được truy cập** hoặc **chứa dữ liệu tĩnh**.
- Sử dụng các thuộc tính **include**, **exclude**, và **max** để kiểm soát số lượng component được cache, giảm thiểu tiêu tốn RAM.

3. Import từng phần của node module cần dùng (Tree-shaking)

- **Tree-shaking** là một kỹ thuật tối ưu hóa trong JavaScript nhằm **loại bỏ các phần mã không được sử dụng** giúp **giảm kích thước** bundle, tăng hiệu suất tải trang.
- **Tree-shaking** hoạt động tốt nhất với các module ES6 sử dụng cú pháp **`import`** và **`export`**. Các module CommonJS (sử dụng require và module.exports) thì không hỗ trợ tốt.

=> DEV cần lưu ý chỉ import những module mà mình cần sử dụng từ thư viện (bao gồm cả thư viện trong node_modules và thư viện do mình tự viết)



4. Bundle và Minify resource

- Cấu hình **Webpack** trong Vue để bundle và minify các resource
- Cấu hình trong file **vue.config.js**
- Cần chú ý **kiểm tra dung lượng các file sau khi bundle**, tránh dung lượng quá lớn ảnh hưởng đến tốc độ tải (Khuyến nghị 1 file bundle < 200KB). Có thể sử dụng `webpack-bundle-analyzer` để kiểm tra dung lượng (Đã đề cập ở slide Code-Splitting)

```
module.exports = {
  // Cấu hình các tùy chọn của Vue CLI
  configureWebpack: {
    // Tùy chỉnh thiết lập cho Webpack
    // Ví dụ: thêm plugins, rules, ...
    plugins: [
      // Thêm một plugin vào Webpack
      new MyCustomPlugin()
    ],
    // Tối ưu hóa cho production
    optimization: {
      minimize: true,
      splitChunks: {
        chunks: 'all',
      },
    },
  },
};
```

Tối ưu code trong Vue

1. Lưu ý khi sử dụng v-if và v-show

- **v-if**: Khi sử dụng `v-if`, Vue sẽ thêm hoặc xóa các phần tử khỏi DOM tùy thuộc vào điều kiện, nếu là false thì phần tử và các thành phần con của nó không tồn tại trong DOM nữa.
- **v-show**: `v-show` thay đổi thuộc tính CSS `display` của phần tử để ẩn hoặc hiển thị nó, thay vì thêm/xóa phần tử khỏi DOM, tức là phần tử luôn tồn tại trong DOM, chỉ thay đổi cách nó được hiển thị.

➡ Khi nào nên sử dụng `v-if`?

- Khi điều kiện hiển thị không thay đổi thường xuyên, hoặc khi muốn kiểm soát việc render các component khi thực sự cần thiết, hoặc khi cần render các component phức tạp/nặng mà tốn tài nguyên.

➡ Khi nào nên sử dụng `v-show`?

- Khi điều kiện hiển thị thay đổi liên tục thì nên dùng v-show để tránh việc tạo lại hoặc phá hủy phần tử liên tục.

1. Lưu ý khi sử dụng v-if và vì v-show

Ví dụ dưới đây để thấy rõ sử dụng `v-if` tối ưu hơn `v-show`, trong vòng for cần hiển thị chi tiết của bản ghi theo từng item, nếu dùng `v-show` thì sẽ tạo ra rất nhiều DOM và chỉ ẩn chúng đi, trong khi dùng `v-if` thì Vue sẽ thêm hoặc xóa phần tử khỏi DOM theo điều kiện và chỉ tồn tại DOM của item được active.

KHÔNG NÊN

```
<li v-for="(item, index) in items" :key="index">
  {{ item.name }}
  <button @click="showDetails(index)">Xem chi tiết</button>
  <div v-show="activeIndex === index">
    Chi tiết: {{ item.details }}
  </div>
</li>
```

NÊN

```
<li v-for="(item, index) in items" :key="index">
  {{ item.name }}
  <button @click="showDetails(index)">Xem chi tiết</button>
  <div v-if="activeIndex === index">
    Chi tiết: {{ item.details }}
  </div>
</li>
```

2. Tránh sử dụng v-for và v-if trên cùng một phần tử

- Sử dụng `v-for` và `v-if` trên cùng một phần tử, Vue sẽ chạy điều kiện `v-if` trong mỗi lần lặp của `v-for` dẫn tới giảm hiệu năng hệ thống.

➡ **Nên xử lý filter danh sách trước khi đưa vào vòng lặp**

✖ KHÔNG NÊN

```
<template>
  <div>
    <ul>
      <li v-for="item in items" :key="item.id" v-if="item.isVisible">
        {{ item.name }}
      </li>
    </ul>
  </div>
</template>
```

✔ NÊN

```
<template>
  <div>
    <ul>
      <li v-for="item in filteredItems" :key="item.id">
        {{ item.name }}
      </li>
    </ul>
  </div>
</template>
```

```
computed: {
  filteredItems() {
    return this.items.filter(item => item.isVisible);
  },
},
```

3. Chú ý khi sử dụng Watch

- **Tránh deep watch** nếu không cần thiết

  Sử dụng watch với `deep: true` để quan sát các đối tượng lồng nhau hoặc mảng có thể gây tốn hiệu năng => **chỉ dùng khi thực sự cần** theo dõi toàn bộ cấu trúc dữ liệu đó

Ví dụ có đối tượng **user**, và chỉ cần theo dõi watch của property *`name`* => không cần deep watch cả đối tượng user mà chỉ cần watch *`user.name`*

KHÔNG NÊN

```
<template>
  <div>
    <p>{{ user.name }}</p>
    <p>{{ user.age }}</p>
  </div>
</template>
```

```
watch: {
  user: {
    handler(newValue) {

    },
    deep: true
  }
}
```

NÊN

```
<template>
  <div>
    <p>{{ user.name }}</p>
    <p>{{ user.age }}</p>
  </div>
</template>
```

```
watch: {
  'user.name': function(newName) {

  },
}
```

4. Tối ưu hóa Computed Properties

Một số lưu ý khi sử dụng Computed Properties để tối ưu hiệu năng và code dễ bảo trì hơn

- Tránh tính toán phức tạp trong computed properties: nếu tính toán qúa phức tạp thì nên dùng methods.
- Computed chỉ nên thực hiện các phép tính thuần túy và trả về giá trị chứ không nên làm các việc khác như: thay đổi trạng thái, cập nhật DOM,...
- Tránh thay đổi giá trị của computed đã được tính toán để đảm bảo tính nhất quán giữa giá trị tính toán và giá trị thực tế của các thuộc tính mà nó dựa vào

4. Tối ưu hóa Computed Properties

Ví dụ khi sử dụng computed chỉ làm việc tính toán chứ không thay đổi trạng thái khác dễ gây side effect và không thay đổi giá trị computed đã được tính toán

✖ KHÔNG NÊN

```
const counter = ref(0);
// Computed property với side effects
const doubleCounter = computed(() => {
  counter.value++;
  return counter.value * 2;
});
const increment = () => {
  counter.value++;
};
// Thay đổi giá trị trực tiếp của computed property (sai)
const mutateDoubleCounter = () => {
  doubleCounter.value = 100; // Sai, không nên làm
};
```

✔ NÊN

```
const counter = ref(0);
// Computed property không có side effects
const doubleCounter = computed(() => {
  return counter.value * 2;
});
const increment = () => {
  counter.value++;
};
// Thay đổi giá trị của counter,
// không thay đổi trực tiếp giá trị của computed
const setCounterTo50 = () => {
  counter.value = 50;
};
```

5. Phân tách Component

- **Không nên thiết kế 1 component quá lớn**, hãy phân tách component thành các component con để có thể tái sử dụng, dễ bảo trì và tối ưu hiệu năng hơn.
- **Nên phân tách Component khi nào?**
  - Khi **component quá lớn**, thực hiện nhiều chức năng khác nhau hoặc có chức năng phức tạp. VD: Component cần mở nhiều popup hoặc có nhiều tab…
  - Khi component có phần chức năng có thể **tái sử dụng cho component khác** được. Ví dụ như form chi tiết nhân viên có thể sử dụng ở tính năng Thêm nhanh, component có phần Header dùng chung cho các form khác thì tách thành 1 component Header riêng.

Ví dụ, có Các màn hình danh mục đều có phần header giống nhau gồm tiêu đề và toolbar giống nhau => Có thể tách component Header.vue để tái sử dụng cho các form

Header.vue

```
<template>
  <header class="header">
    <h1>{{ title }}</h1>
    <div class="toolbar">
      <button v-if="showAdd" @click="onAdd">Add</button>
      <button v-if="showEdit" @click="onEdit">Edit</button>
      <button v-if="showDelete" @click="onDelete">Delete</button>
    </div>
  </header>
</template>
```

EmployeeList.vue

```
<template>
  <div>
    <Header
      title="Danh sách nhân viên"
    />
    <div class="employee-list">
    </div>
  </div>
</template>
```

CustomerList.vue

```
<template>
  <div>
    <Header
      title="Danh sách khách hàng"
    />
    <div class="customer-list">
    </div>
  </div>
</template>
```

5. Sử dụng Async Components

- **Async components:** là kỹ thuật cho phép tải các component chỉ khi thực sự cần thiết, giúp giảm thời gian tải trang ban đầu và tối ưu hóa trải nghiệm người dùng.
- Cách sử dụng: Import `defineAsyncComponent` từ Vue, sử dụng `defineAsyncComponent` để define component sẽ được tải async.

```
import {defineAsyncComponent, ref} from 'vue';

const refTab = ref(null);

const PatternEstimateExpenseRegularEnact = defineAsyncComponent(() =>
  import('@/views/enactDictionary/patternEstimate/PatternEstimateExpenseRegularEnact.vue'),
);
const RevenueBudgetEnact = defineAsyncComponent(() =>
  import('@/views/enactDictionary/dictionary/RevenueBudgetEnact.vue'),
);
```

Tối ưu code Javascript

1. Sử dụng Debounce và Throttle

- Sử dụng **Debounce** và **Throttle** để tối ưu hóa các sự kiện người dùng trong JavaScript, đặc biệt là các sự kiện có tần suất cao như `scroll`, `resize`, `keyup`, `mousemove`, `mouseup`,...

Debound: đảm bảo rằng một hàm chỉ được thực thi sau một khoảng thời gian nhất định kể từ lần gọi cuối cùng, thường được sử dụng để giảm số lần gọi hàm khi sự kiện liên tục được kích hoạt như khi người dùng gõ phím hoặc thay đổi kích thước cửa sổ trình duyệt.

Ví dụ: sử dụng **Debound** cho thao tác tìm kiếm mỗi khi người dùng gõ phím trên ô input, nhưng chỉ sau khi người dùng ngừng gõ một khoảng thời gian 300ms thì mới xử lý tìm kiếm

```
<template>
  <div>
    <input type="text" v-model="searchText" @input="handleInput" />
    <ul>
      <li v-for="item in filteredResults" :key="item">{{ item }}</li>
    </ul>
  </div>
</template>

import debounce from 'lodash.debounce';

const searchText = ref('');
const handleInput = debounce(() => {
  console.log('Searching for:', searchText.value);
}, 500);

const filteredResults = computed(() => {
  return resultDatas.value.filter((item) =>
    item.toLowerCase().includes(searchText.value.toLowerCase())
  );
});
```

Throttle: đảm bảo rằng một hàm được thực thi không quá một lần trong một khoảng thời gian nhất định, thường được sử dụng để kiểm soát tần suất thực thi của các hàm được kích hoạt liên tục như khi người dùng cuộn trang hoặc di chuyển chuột.

Ví dụ: sử dụng **Throttle** cho thao tác ghi log lại vị trí cuộn của trang nhưng chỉ cập nhật mỗi 200ms một lần để tránh việc hàm xử lý sự kiện cuộn trang được gọi quá thường xuyên

```
<template>
  <div id="appdemo" @scroll="handleScroll" ref="scrollContainer">
    <div class="content-box">
      <p>Scroll down to see the throttling in action.</p>
      <p v-for="n in 50" :key="n">Content {{ n }}</p>
    </div>
  </div>
</template>

import throttle from 'lodash.throttle';

const handleScroll = throttle(() => {
  console.log(`Scroll position: ${window.scrollY}`);
}, 200);

onMounted(() => {
  if (scrollContainer.value) {
    scrollContainer.value.addEventListener('scroll', handleScroll);
  }
});

onUnmounted(() => {
  if (scrollContainer.value) {
    scrollContainer.value.removeEventListener('scroll', handleScroll);
  }
});
```

2. Giảm Thiểu Số Lần Truy Cập và Thay Đổi DOM

Hiện nay các Framework JS như Vue, Angular,... đều đã hỗ trợ mạnh mẽ làm việc với DOM thông qua các directive và bindings, do đó cần **hạn chế sử dụng JS trực tiếp để thao tác DOM.**

Nếu trường hợp bắt buộc phải thao tác với DOM (vd: xử lý control chung) thì cần lưu ý như sau:

- **DocumentFragment:** Sử dụng **`DocumentFragment`** để tạo và thay đổi các phần tử DOM trước khi thêm chúng vào DOM thật. Điều này giúp giảm số lần truy cập và thay đổi trực tiếp trên DOM thật.

```
const fragment = document.createDocumentFragment();

for (let i = 0; i < 1000; i++) {
    const div = document.createElement('div');
    div.textContent = `Item ${i}`;
    fragment.appendChild(div);
}

document.getElementById('container').appendChild(fragment);
```

- **Caching DOM Elements**: Lưu trữ các phần tử DOM cần truy cập nhiều lần vào biến để tránh truy cập lại DOM nhiều lần.

KHÔNG NÊN

```
document.getElementById('my-element').style.color = 'blue';
document.getElementById('my-element').style.backgroundColor = 'red';
```

NÊN

```
const element = document.getElementById('my-element');
element.style.color = 'blue';
element.style.backgroundColor = 'red';
```

3. Tối ưu vòng lặp

- **Sử dụng vòng lặp `for` thay vì `while`,** vì vòng lặp `for` cho phép khai báo và cập nhật biến điều khiển ngay trong vòng lặp. Điều này tránh tình huống lặp vô tận và làm cho mã dễ đọc hơn.

✖ KHÔNG NÊN

```
const arrayDatas = [[1, 2, 3],[4, 5, 6],[7, 8, 9]
];
let sum = 0;
let i = 0;
let j = 0;

// Sử dụng vòng lặp while để tính tổng
while (i < arrayDatas.length) {
  let j = 0;
  while (j < arrayDatas[i].length) {
    sum += arrayDatas[i][j];
    j++;
  }
  i++;
}
console.log(`sum = ${sum}`);
```

✔ NÊN

```
//Mảng 2 chiều
const arrayDatas = [[1, 2, 3],[4, 5, 6],[7, 8, 9]
];

let sum = 0;

// Sử dụng vòng lặp for để tính tổng
for (let i = 0; i < arrayDatas.length; i++) {
  for (let j = 0; j < arrayDatas[i].length; j++) {
    sum += arrayDatas[i][j];
  }
}
console.log(`sum = ${sum}`);
```

3. Tối ưu vòng lặp

- **Giảm thiểu điều kiện tính toán không cần thiết trong vòng lặp,** để tránh thực hiện lại các phép tính này nhiều lần.

KHÔNG NÊN

```
console.time('Loop time');
const numbers = Array.from({ length: 1000 },
                           (_, index) => index + 1);
for (let i = 0; i < numbers.length; i++) {
  if (numbers.length > 100) {
    console.log(numbers[i]);
  }
}
console.timeEnd('Loop time');
```

Thời gian thực hiện vòng lặp: `Loop time: 135.139892578125 ms`

NÊN

```
console.time('Loop time');
const numbers = Array.from({ length: 1000 },
                           (_, index) => index + 1);
const numberLength = numbers.length;
if (numberLength > 100) {
  for (let i = 0; i < numbers.length; i++) {
    console.log(numbers[i]);
  }
}
console.timeEnd('Loop time');
```

Thời gian thực hiện vòng lặp: `Loop time: 102.166015625 ms`

3. Tối ưu vòng lặp

- **Sử dụng Array Methods** như **`map`, `filter`, `reduce`** thường được tối ưu hóa bởi JavaScript engine, thường có hiệu năng tốt hơn và code cũng clean, dễ đọc hơn

KHÔNG NÊN

```
const numbers = [1, 2, 3, 4, 5];
const doubled = [];

for (let i = 0; i < numbers.length; i++) {
  doubled.push(numbers[i] * 2);
}

console.log(doubled); // [2, 4, 6, 8, 10]
```

NÊN

```
const numbers = [1, 2, 3, 4, 5];
const doubled = numbers.map(number => number * 2);

console.log(doubled); // [2, 4, 6, 8, 10]
```

3. Tối ưu vòng lặp

- **Hạn chế sử dụng nhiều vòng lặp lồng nhau**, vì độ phức tạp thời gian của các vòng lặp lồng nhau thường là O(n^2) hoặc cao hơn, do đó cố gắng tối ưu để chỉ nên sử dụng 1 vòng lặp giúp hiệu năng tốt hơn.

KHÔNG NÊN

```
const array1 = generateRandomArray(1000, 1, 10000);
const array2 = generateRandomArray(1000, 1, 10000);

console.time('Loop time');
const commonElements = [];
for (let i = 0; i < array1.length; i++) {
  for (let j = 0; j < array2.length; j++) {
    if (array1[i] === array2[j]) {
      commonElements.push(array1[i]);
    }
  }
}
```

Thời gian thực hiện vòng lặp: `Loop time: 9.58984375 ms`

NÊN

```
const array1 = generateRandomArray(1000, 1, 10000);
const array2 = generateRandomArray(1000, 1, 10000);

console.time('Loop time');
const set1 = new Set(array1);
const commonElements = array2.filter((item) => set1.has(item));
console.log(commonElements);
console.timeEnd('Loop time');
```

Thời gian thực hiện vòng lặp: `Loop time: 1.161865234375 ms`

4. Tối ưu Asynchronous Code

- **Chạy song song các tác vụ không đồng bộ**, nếu các tác vụ không phụ thuộc lẫn nhau thì hãy chạy chúng song song để tiết kiệm thời gian.

Ví dụ dưới đây cần lấy 3 dữ liệu data1, data2, data3 không phụ thuộc nhau, sau khi đã có đủ cả 3 dữ liệu thì sẽ xử lý tiếp công việc

KHÔNG NÊN

```
async function fetchMultipleData() {
  const data1 = await fetchData1();
  const data2 = await fetchData2();
  const data3 = await fetchData3();
  // Sử dụng data1, data2, data3
}
```

NÊN

```
async function fetchMultipleData() {
  const [data1, data2, data3] = await Promise.all([
    fetchData1(),
    fetchData2(),
    fetchData3()
  ]);
  // Sử dụng data1, data2, data3
}
```

4. Tối ưu Asynchronous Code

- **Tối ưu khi sử Dụng `async`/`await` trong vòng lặp,** hãy hạn chế dùng await trực tiếp trong vòng lặp nếu kết quả trong vòng lặp không phụ thuộc vào nhau, nếu dùng await sẽ khiến các tác vụ sẽ chạy tuần tự gây ảnh hưởng đến hiệu năng. Thay vào đó, thu thập tất cả các promises trước và đợi chúng song song

Ví dụ dưới đây vòng lặp cần gọi hàm **`processItem`** để xử lý từng item khi duyệt vòng lặp, trong đó kết quả hàm **`processItem`** của từng item không phụ thuộc vào nhau

KHÔNG NÊN

```
async function processArray(array) {
  for (const item of array) {
    await processItem(item); // Chạy tuần tự
  }
}
```

NÊN

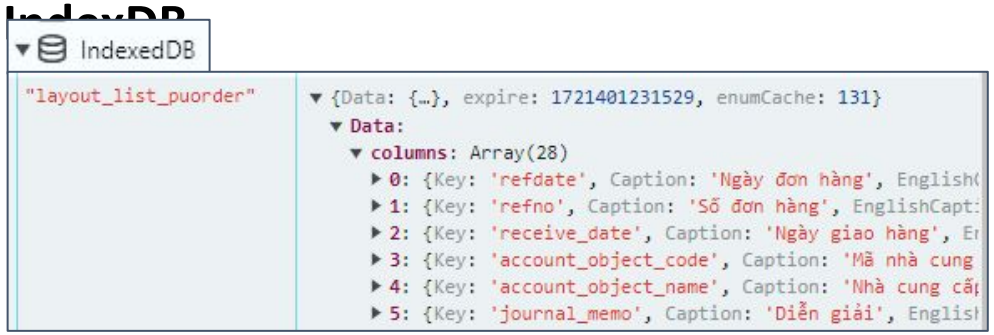
```
async function processArray(array) {
  const promises = array.map(item => processItem(item));
  await Promise.all(promises); // Chạy song song
}
```

Tối ưu request API

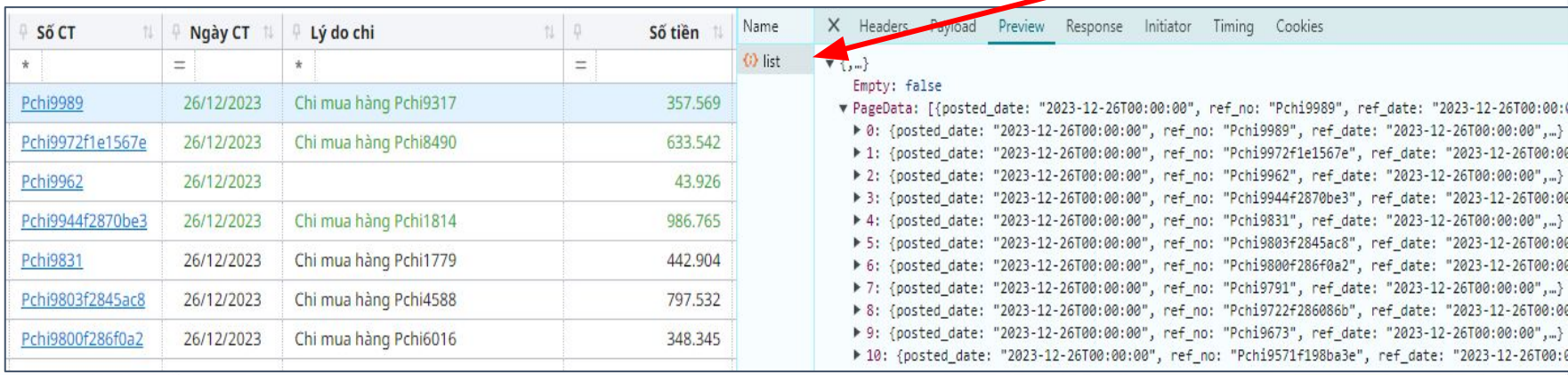
1. Cache dữ liệu trên Client

- Trong nội dung đào tạo Cache cho LTV đã nói về các loại Cache trên client và cách dùng của từng loại (IndexDB, LocalStorage, SessionStorage) **=> Cần áp dụng cache client để lưu lại những dữ liệu thường dùng mà ít thay đổi nhằm tăng hiệu năng lấy dữ liệu và giúp tối ưu số request API gọi lên Server.**
- Để lưu cache dữ liệu nên dùng **IndexDB**

Ví dụ về **cache dữ liệu layout** (cấu hình các cột hiển thị cho các màn hình danh sách) vào **IndexDB**
=> Mỗi lần vào màn hình danh sách chỉ cần gọi API lấy dữ liệu mà **không cần phải gọi API lấy danh sách layout** nữa.



Đã có cache Layout nên khi xem danh sách không lấy layout nữa mà chỉ gọi API lấy dữ liệu



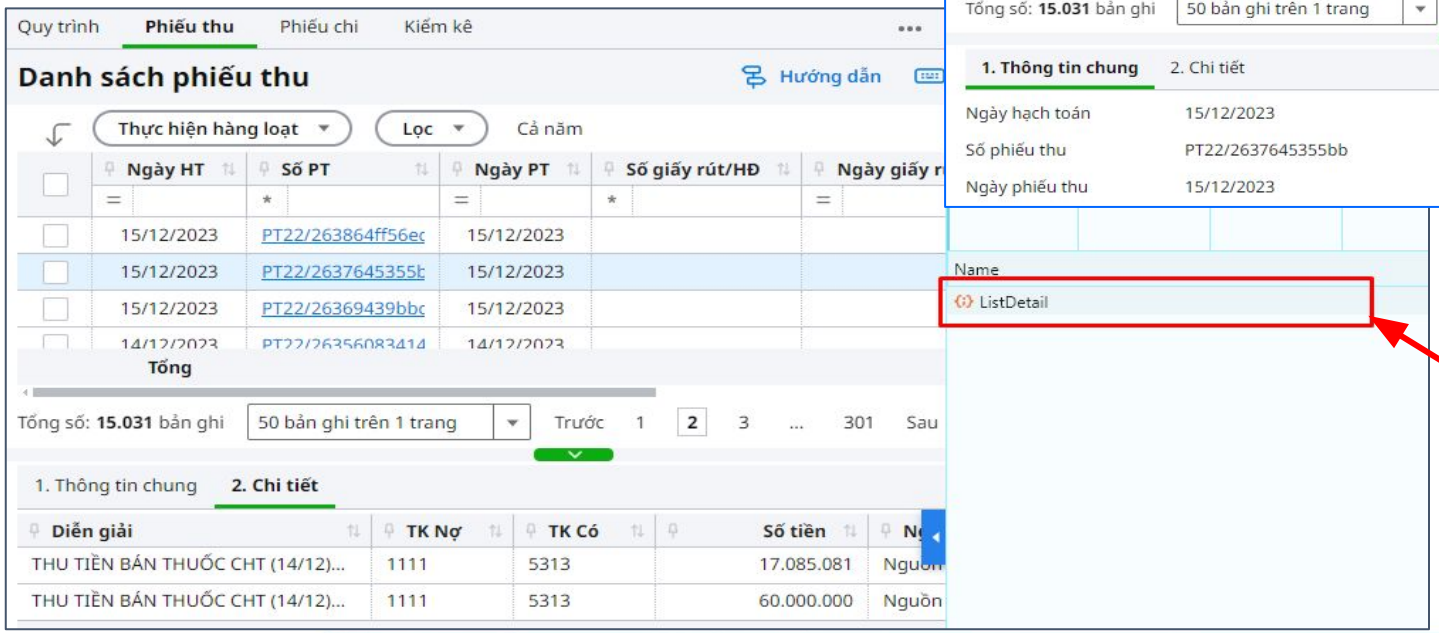
2. Load On-Demand

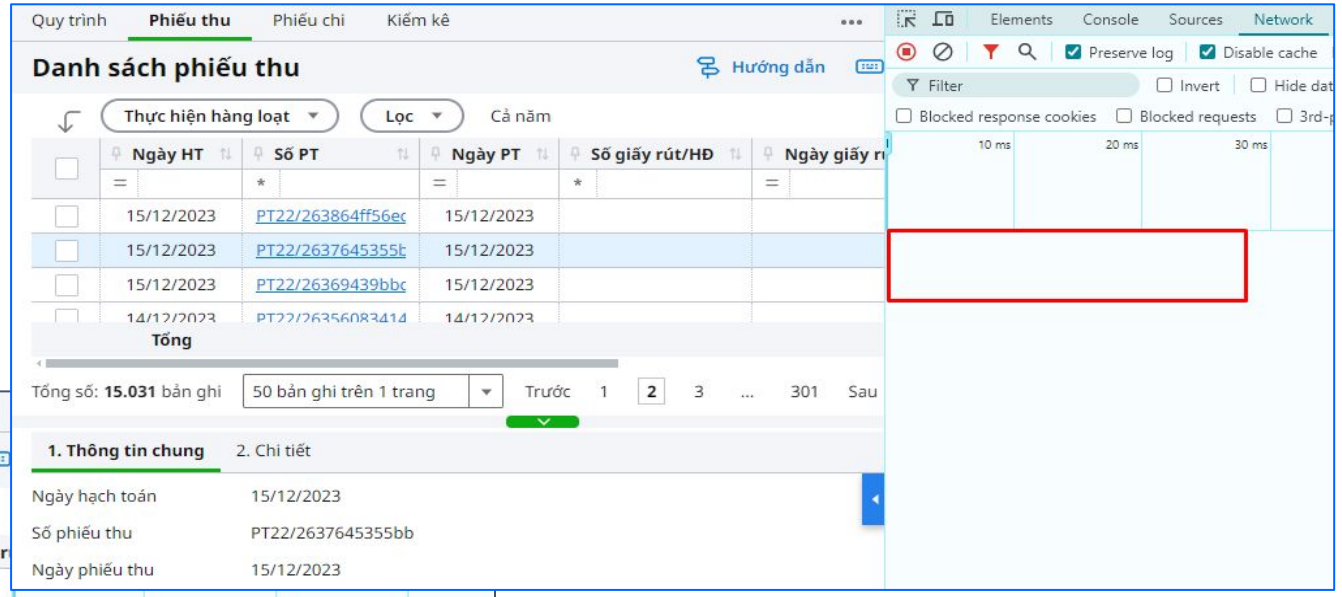
- **Load On-Demand:** là kỹ thuật xử lý **lấy dữ liệu khi có nhu cầu** dùng đến, giúp tối ưu hóa request API gọi lên server.
- Một số trường hợp **thường cần xử lý** Load On-Demand như sau:
  - Sử dụng **Tab**: Chỉ load dữ liệu khi click vào tab, khi chuyển sang tab khác quay lại tab đã click thì không load lại dữ liệu nữa.
  - Sử dụng **Combobox**: Với các form nhập liệu có Combobox cần lấy dữ liệu từ Server thì chỉ nên load dữ liệu Combobox khi xổ combo ra.
  - Màn hình nhiều view **có scroll** kéo xuống để xem các view bên dưới (Ví dụ Dashboard): Chỉ lấy dữ liệu cho các view nhìn thấy ở trên màn hình, khi scroll xuống dưới thì mới load tiếp dữ liệu cho view bên dưới.

2.1 Xử lý Load On-Demand với Tab

- **Ví dụ màn hình dùng Tab cần xử lý Load On-Demand**

Khi click vào 1 dòng phiếu thu để xem thông tin chi tiết gồm 2 tab: *1. Thông tin chung* và *2. Chi tiết* => xử lý load on-demand là khi chưa click vào tab 2. Chi tiết thì chưa gọi API lấy dữ liệu.



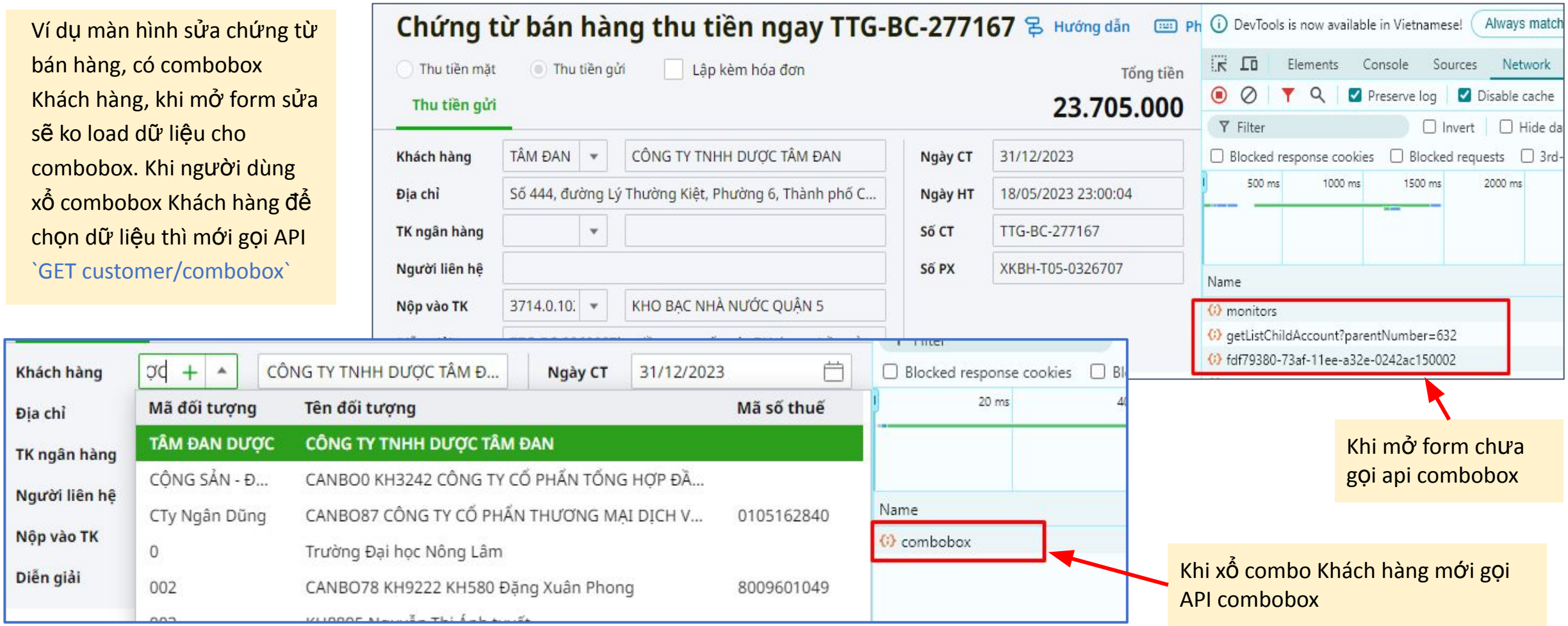


Khi click tab 2. Chi tiết mới gọi api lấy dữ liệu cho grid chi tiết

2.2. Xử lý Load On-Demand với Combobox

- **Ví dụ màn hình dùng Combobox cần xử lý Load On-Demand**

Ví dụ màn hình sửa chứng từ bán hàng, có combobox Khách hàng, khi mở form sửa sẽ ko load dữ liệu cho combobox. Khi người dùng xổ combobox Khách hàng để chọn dữ liệu thì mới gọi API `GET customer/combobox`



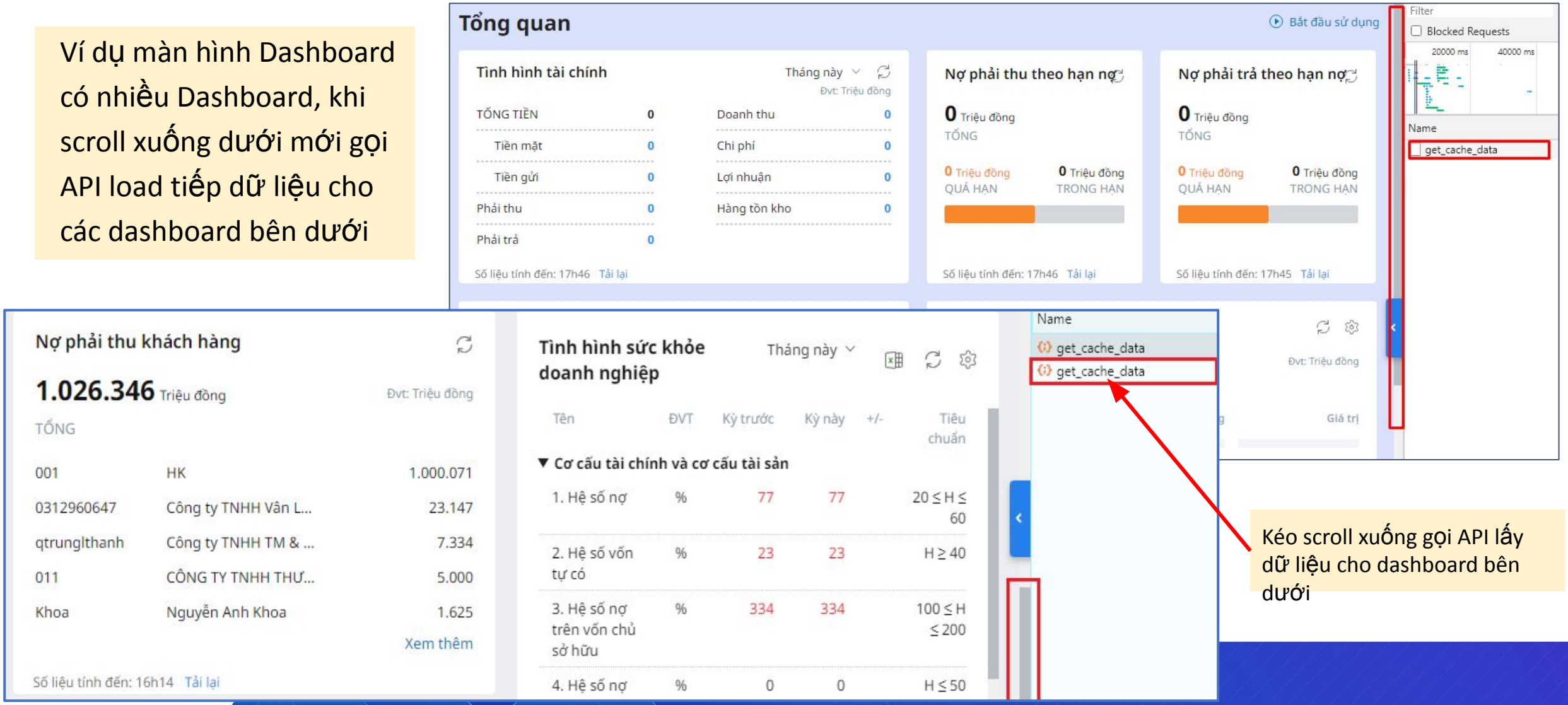
Khi mở form chưa gọi api combobox

Khi xổ combo Khách hàng mới gọi API combobox

2.3. Xử lý Load On-Demand màn hình nhiều view có scroll

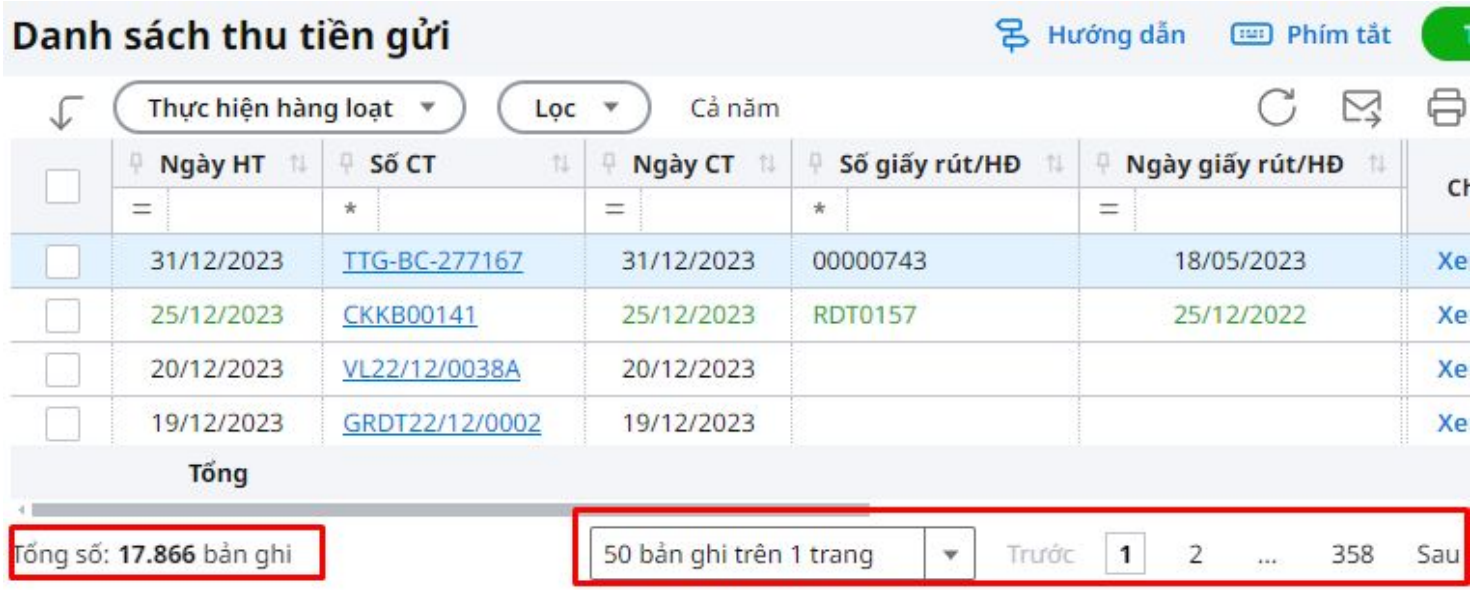
- **Ví dụ màn hình nhiều view có scroll cần xử lý Load On-Demand**

Ví dụ màn hình Dashboard có nhiều Dashboard, khi scroll xuống dưới mới gọi API load tiếp dữ liệu cho các dashboard bên dưới



3. Sử dụng Paging Data

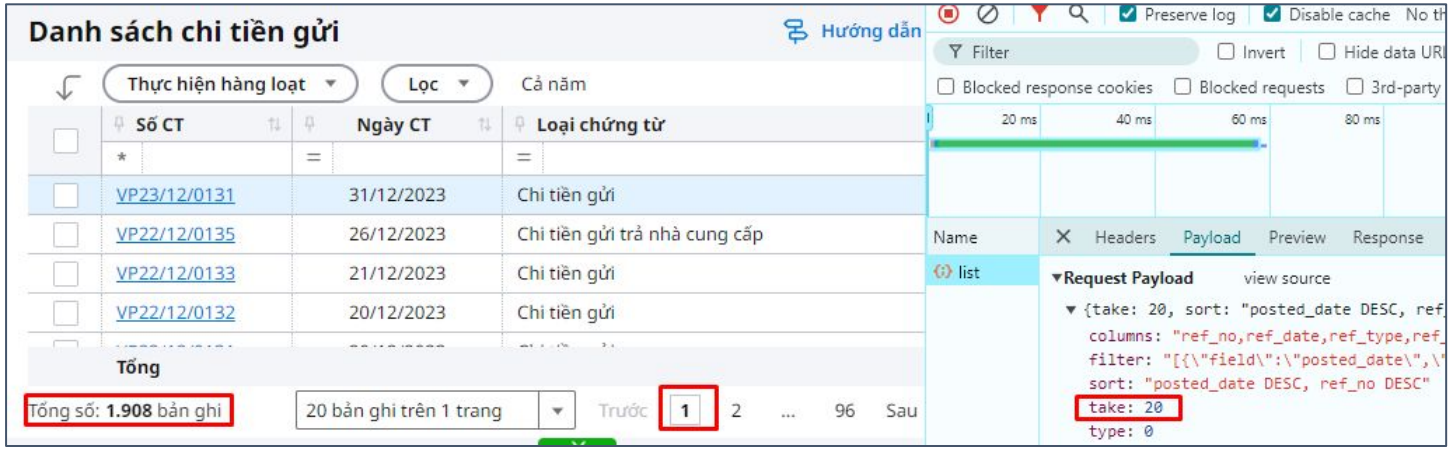
- **Paging Data:** là kỹ thuật phân trang dữ liệu nhằm chia nhỏ dữ liệu lớn thành các phần nhỏ hơn, được gọi là các "trang". Mỗi trang chứa một lượng dữ liệu nhất định và người dùng có thể chọn các trang để xem dữ liệu cần. Kỹ thuật này giúp cải thiện hiệu suất và trải nghiệm người dùng bằng cách giảm lượng dữ liệu cần tải và hiển thị tại một thời điểm.
- Có 2 kỹ thuật Paging:
  - **Paging Server**
  - **Paging Local**

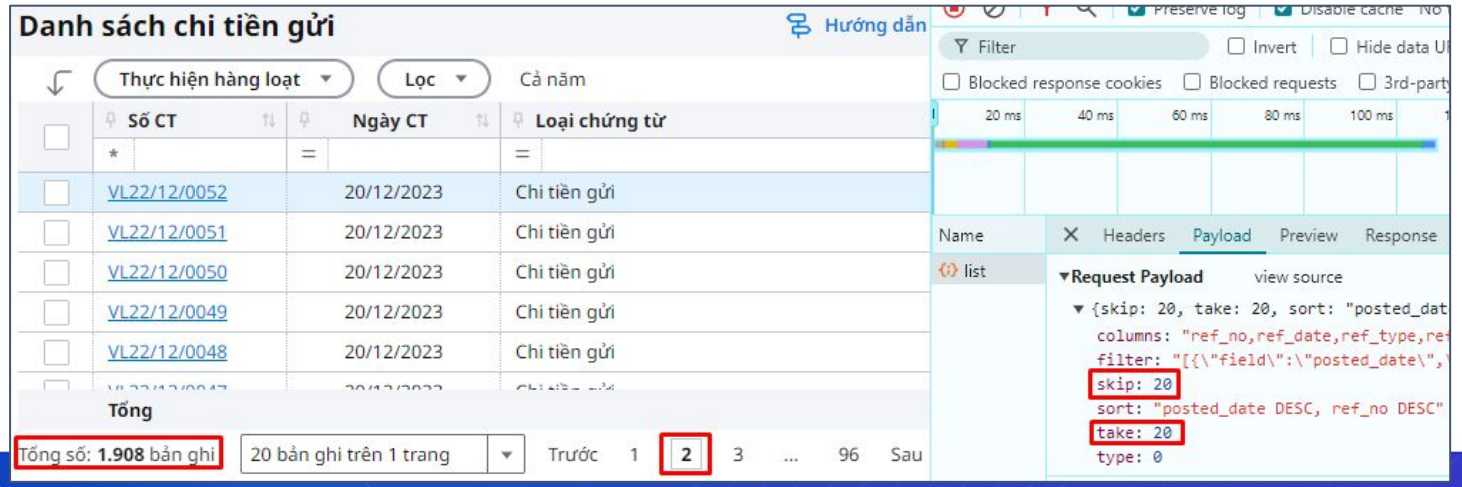


3.1 Paging Server

- **Paging Server:** là giải pháp client gọi API lên server lấy dữ liệu từng trang, và chỉ render ra màn hình dữ liệu của trang trả về => giúp **server giảm thời gian xử lý lấy dữ liệu** và **giảm dung lượng data trả về**, giúp **client render nhanh hơn**.
- Nên áp dụng **Paging Server** khi:
  - Áp dụng các màn hình dạng danh sách, combobox
  - Dữ liệu > 100 bản ghi

Ví dụ màn hình danh sách Chi tiền gửi có 1908 bản ghi, client sẽ **không gọi API lấy tất cả bản ghi** về mà sẽ t**ạo ra thanh phân trang**, mỗi trang chỉ lấy 20 bản ghi, click trang nào sẽ gọi API lấy dữ liệu trang đó và render ra gridview.



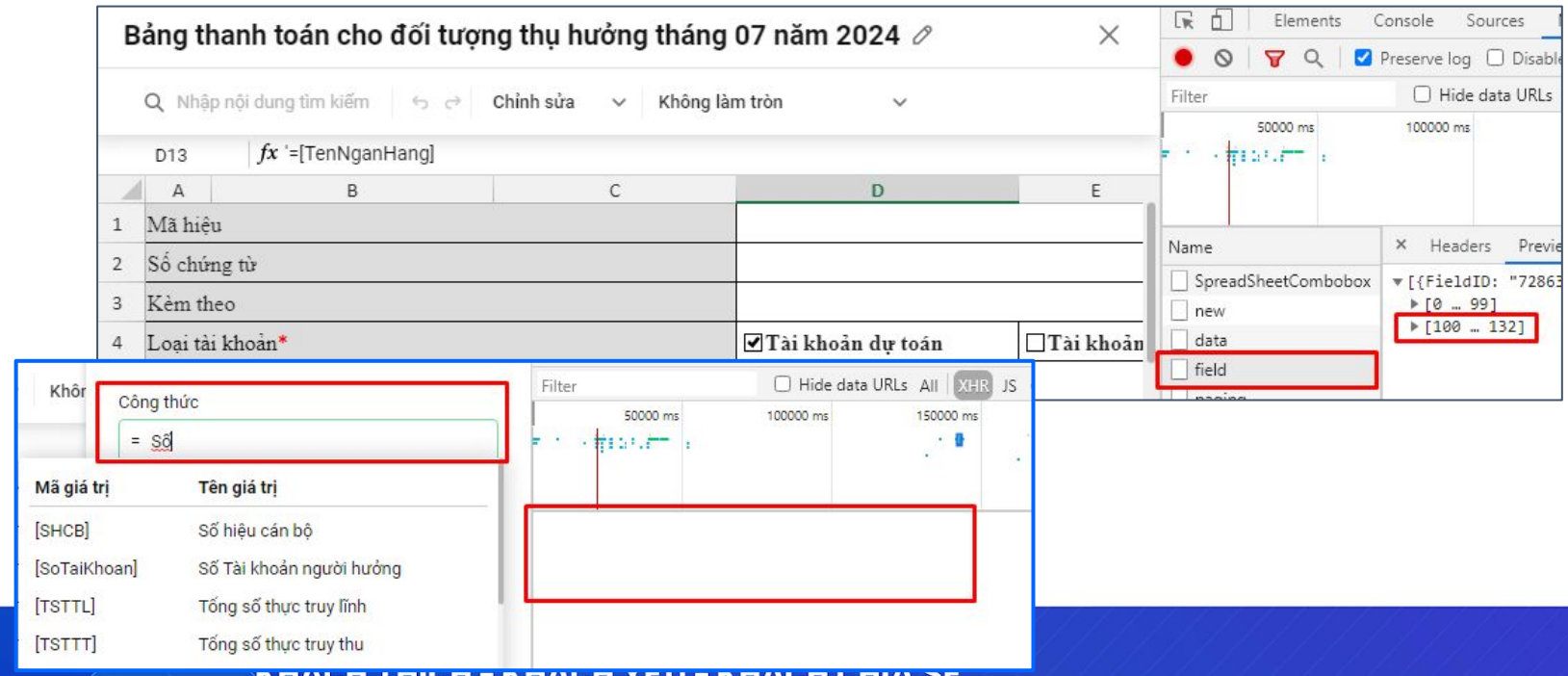


3.2 Paging Local

- **Paging Local:** là giải pháp **lấy dữ liệu từ server về 1 lần**, sau đó được xử lý **phân trang trên client** để **giảm thời gian render trên client** và giảm độ phức tạp cho code xử lý nghiệp vụ.
- Nên áp dụng **Paging local** khi:
  - Tổng số dòng **dữ liệu đủ nhỏ**, thường là **dưới 100 bản ghi**

    *(Lưu ý **cần đánh giá đúng số lượng dữ liệu** tránh dữ liệu Khách hàng này thì ít nhưng Khách hàng kia lại rất nhiều và **cân nhắc với trường hợp > 100 bản ghi** mà vẫn cần dùng giải pháp paging local thì cần **đánh giá hiệu năng API** và **dung lượng response** trả về)*
  - Màn hình danh sách hình cây, màn hình grid editor (cho phép nhập liệu trực tiếp trên grid) hoặc màn hình có combobox cần thao tác xổ ra nhiều lần để chọn dữ liệu.

Ví dụ màn hình lập Bảng thanh toán có combo Công thức, khi mở màn hình sẽ gọi API `GET /field` load toàn bộ công thức (132 bản ghi). Khi xổ combo Công thức để filter, chọn công thức, scroll xuống thì không phải gọi api nữa



3.3 Một số giải pháp xử lý Paging ở Client

- **Danh sách dạng Grid**: dùng thanh toolbar paging
- **Combobox**: sử dụng **Virtual Scroll** (scroll đến cuối trang mới load data), không cần kiểm tra TotalCount (tổng số dòng dữ liệu có ở Server), chỉ cần xác định PageSize (Số bản ghi cần lấy trên 1 trang) để kiểm tra nếu dữ liệu lấy về < PageSize thì không cần gọi API lấy dữ liệu nữa
- **Danh sách dạng cây (Tree)**: Có một số cách như sau:
  - Sử dụng **Virtual Scroll** để phân trang và có thể kết hợp với **Virtual Render** để xử lý **chỉ render những item đang được hiển thị** để tối ưu hiệu năng render, tránh tốn Memory trình duyệt
  - Sử dụng **Lazy Load theo node cha**: chỉ load dữ liệu node cha trước, khi mở rộng (expand) node cha thì mới load dữ liệu các node con.
  - **Tối ưu render ít cấp nhất có thể**: danh sách dạng cây thường dữ liệu có nhiều cấp, để tối ưu thời gian render thì có thể chỉ render 1 hoặc 2 cấp, khi expand node thì mới render cấp tiếp theo.

4. Tránh duplicate request

- Quản lý việc **đăng ký** và **hủy sự kiện** khi không dùng

Trong Vue3, sử dụng các hooks `**onMounted**` và `**onUnmounted**` để quản lý sự kiện

```
onMounted(() => {
  document.addEventListener('click', documentClick);
});

onUnmounted(() => {
  document.removeEventListener('click', documentClick);
});
```

Trường hợp tự quản lý add sự kiện và xóa sự kiện sau khi xử lý xong.

```
button.addEventListener('click', handleEvent);

const handleEvent = () => {
  console.log('Event handled');
  // Thực hiện logic tại đây

  button.removeEventListener('click', handleEvent);
};
```

- Đảm bảo **không đăng ký sự kiện nhiều lần**

Có thể áp dụng một số kỹ thuật để tránh add nhiều sự kiện như: sử dụng **Debounce/Throttle**, sử dụng **cờ** đánh dấu, sử dụng **v-once** hoặc `**.once**` Modifier với các sự kiện chỉ sử dụng 1 lần

Ví dụ sử dụng cờ đánh dấu để quản lý việc add sự kiện

```
setup() {
  const buttonRef = ref(null);
  let isEventAdded = false;
  const handleClick = () => {
    console.log('Button clicked');
    // Thực hiện logic tại đây
  };
  onMounted(() => {
    if (buttonRef.value && !isEventAdded) {
      buttonRef.value.addEventListener('click', handleClick);
      isEventAdded = true;
    }
  });
```

- **Luôn mở DevTools, tab Network** để kiểm tra lại phần mình thi công có bị duplicate request ko

5. Tối ưu request param khi gọi API

- Tối ưu request param khi gọi API lên server giúp **giảm request size**, **tránh lãng phí băng thông**, **giảm thời gian phản hồi** và tải trang nhằm **tăng trải nghiệm** cho người dùng.
  - *Chỉ gửi những tham số thật sự cần thiết để server xử lý request.*
  - *Đặt tên tham số ngắn gọn nhưng đủ rõ nghĩa.*
  - *Áp dụng DTO Pattern, tạo các DTO tham số theo yêu cầu của request.*

Ví dụ, client cần gọi api lấy danh sách chứng từ chi tiết theo refid và reftype của chứng từ. Do đó cần tạo đối tượng lấy dữ liệu là ***`VoucherDetailParamDTO`*** chỉ bao gồm **2 tham số cần truyền lên server là *`refid`* và *`reftype`*** chứ **KHÔNG** sử dụng sẵn đối tượng `VoucherList` đã có.

TẠO class `VoucherDetailParamDTO` chứ **KHÔNG** sử dụng class `VoucherList` có sẵn

```
export default class VoucherDetailParamDTO {
  constructor(refid, reftype) {
    this.refid = refid;
    this.reftype = reftype;
  }
}
```

KHÔNG dùng class có sẵn này

```
class VoucherList {
  constructor(refid, reftype, refno, refdate) {
    this.refid = refid;
    this.reftype = reftype;
    this.refno = refno;
    this.refdate = refdate;
  }
}
```

```
async getVoucherDetails() {
  const voucherParam = new VoucherDetailParamDTO('645355bb-f539-4e3e-8399-c6a2b69ee2ac', 101);
  return await axios.post('/api/voucher/details', voucherParam);
}
```

6. Sử dụng bất đồng bộ gọi đồng thời nhiều API

- Trong trường hợp một màn hình **cần xử lý nhiều công việc lấy dữ liệu trên Server** nhưng **không cần tuần tự mà có thể xử lý song song** thì có thể sử dụng bất đồng bộ để gọi đồng thời nhiều API

  => Giúp giảm thời gian chờ ở client.

Ví dụ, xử lý màn hình danh sách Nhân viên, gridview cần gọi 2 api: `*loadData*` để lấy dữ liệu bind vào gridview và `*loadDataSummary*` để lấy phần dữ liệu tổng hợp bind vào cuối gridview. 2 Việc này là độc lập mà không cần chờ đợi kết quả của nhau

```
refresh() {
  const me = this;
  let param = me.lastParam || {
    skip: 0,
    take: me.gridInfo.pageSize
  };

  me.loadData(param);
  me.loadDataSumary(param);
},
```

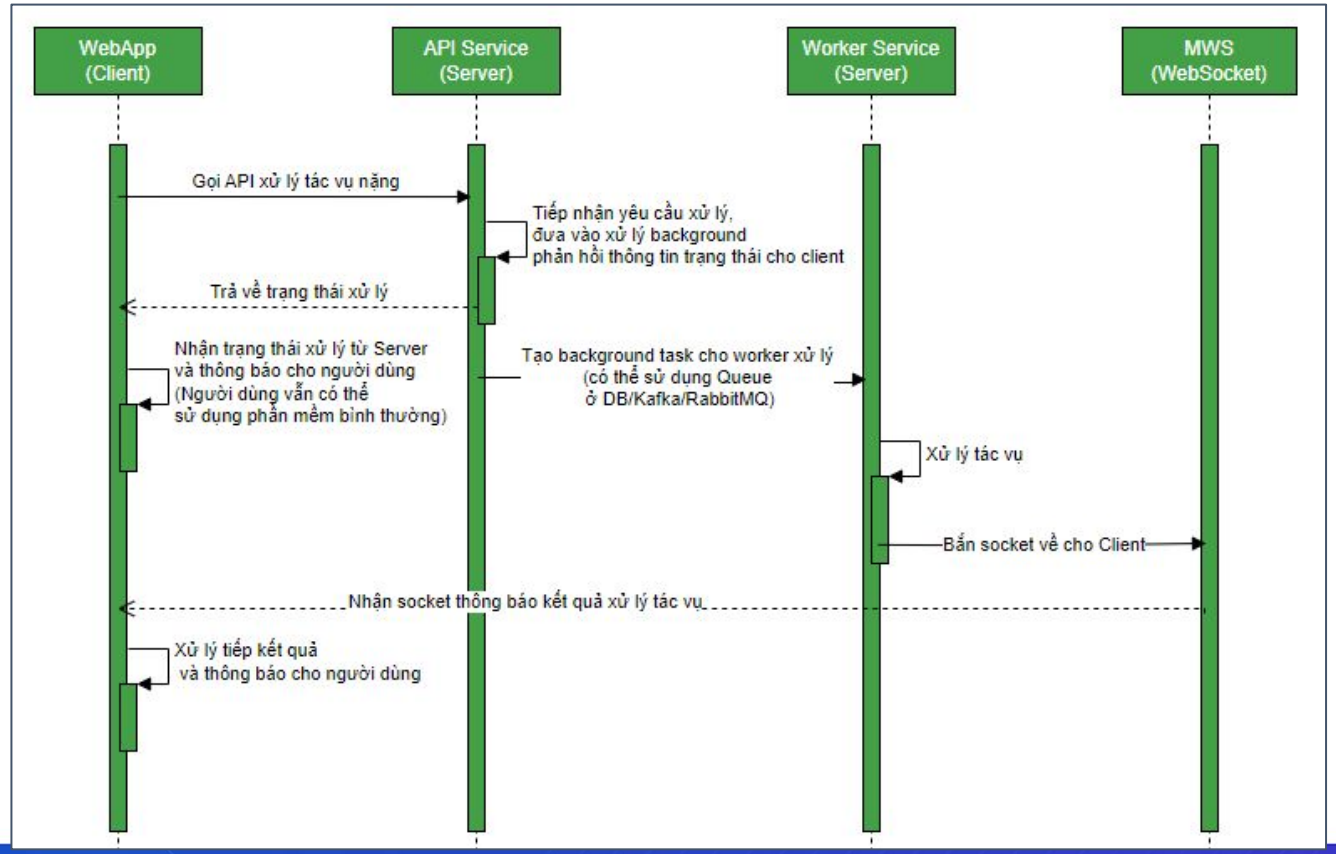
```
async loadData(context, payload) {
  //xử lý load data
},

async loadSummaryData(context, payload) {
  //xử lý load summary data
},
```

7. Xử lý Asynchronous ở Client với các tác vụ nặng

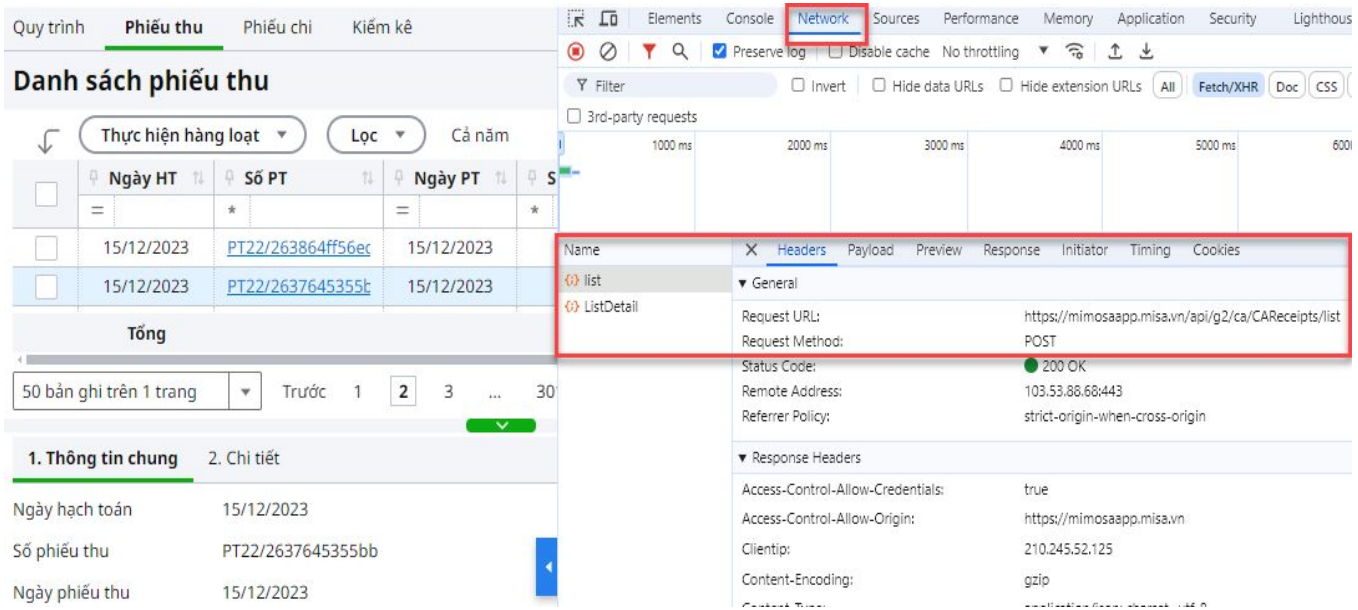
- Trong trường hợp client có yêu cầu server xử lý tác vụ nặng, thời gian chờ server phản hồi về lâu **thì có thể áp dụng pattern xử lý asynchronous (bất đồng bộ) ở client để tăng trải nghiệm cho người dùng.** Thường **áp dụng cho các tác vụ nặng** như: Xuất khẩu, In báo cáo, Ghi sổ hàng loạt,...

- **Pattern xử lý asynchronous** ở client như sau:
  - Client gửi yêu cầu lên Server và thông báo cho người dùng trạng thái đang xử lý.
  - Server có cơ chế xử lý background job và phản hồi về cho client qua WebSocket.
  - Client nhận phản hồi từ Server qua WebSocket và thông báo cho người dùng kết quả xử lý.



8. Sử dụng DevTools để kiểm tra kết quả khi thi công

- Luôn sử dụng **DevTools**, tab **Network** để kiểm tra kết quả khi thi công nhằm đảm bảo đã tối ưu request api, sửa lại ngay **tránh sau phải rework sẽ mất thời gian** hơn rất nhiều.
- **Checklist khi kiểm tra:**
  - *Có duplicate request API không?*
  - *Có gọi thừa request API mà chưa dùng đến không?*
  - *Có truyền thừa tham số lên API không?*
  - *Có trả về thừa trường dữ liệu không?*
  - *Đã xử lý load ondemand chưa? (Lưu ý các màn hình tab, màn hình detail có nhiều combobox, màn hình có nhiều view cần scroll,...)*
  - *Đã xử lý Paging (phân trang dữ liệu) chưa?*
  - *Có thể cache được dữ liệu trên client để tránh gọi API nhiều lần không?*



MISA®
TIN CẬY - TIỆN ÍCH - TẬN TÌNH

THANK YOU!

KHÁCH THÍCH - KHÁCH YÊU - KHÁCH CHIA SẺ